

Abstract

Medical professionals look at x-ray images for the diagnosis of bone injuries, which can sometimes be difficult to interpret. This report focuses on the use of Machine Learning and Deep Learning methods to identify the grade of knee injury presented in an x-ray image, with the objective of producing a model that can effectively assist doctors in diagnosis. Three models have been created and compared to determine the best approach for the problem; a multiclass perceptron, a Convolutional Neural Network (CNN) trained from scratch, and a CNN using shallow transfer learning and fine tuning. The most successful model was found to be the transfer learning. This produced an average accuracy of 0.69 for classification over 5 classes. This indicates that Deep Learning models can be effective in assisting diagnosis.

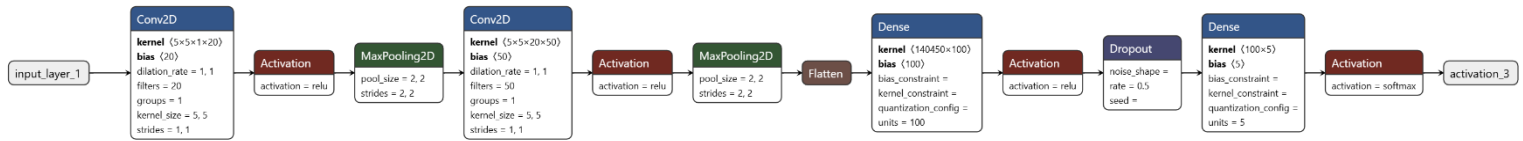
Introduction and Related Work

Machine learning is an umbrella term for many different techniques. Identifying a knee injury grade is a multi-class classification problem. There are several techniques for this, which can be classed as either machine learning or deep learning. Traditional machine learning is the use of algorithms that learn from data and improve with experience. It works effectively with small datasets and is faster to train and less resource intensive. However, accuracy depends on the quality of the chosen features and the algorithm used. Deep learning is a subset of machine learning that uses multi-layered neural networks with hidden layers. These networks can learn complex patterns, such as those found in images, but typically requires larger datasets. Three approaches were chosen for comparison. The first is a multiclass perceptron classifier. This is a supervised learning algorithm that uses a basic neural network to identify features of a class, without using any hidden layers or convolution. The second algorithm tested was CNN trained from scratch. A CNN is a type of deep neural network which is designed to process visual and spatial data and uses a small matrix to detect edges, textures, and objects, a technique known as convolution. This model was chosen for its effectiveness in handling image data. The third method used was shallow transfer learning and fine tuning with a pretrained CNN. Shallow transfer learning is the process of taking a pre-trained CNN as the model's convolutional base and then placing an appropriate classifier at the end of the model to fit the use case. The convolutional base is then frozen, and the classifier is trained on the data specific to the use cases. This provides a model that is both trained on a large dataset and designed for the problem at hand.

Method

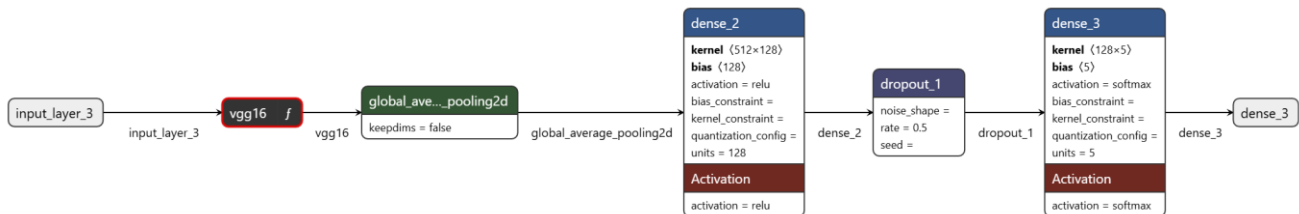
The CNNs were constructed according to the following pipeline diagrams:

CNN:



It begins with an Input layer of size 224x224, and 2 convolutional layers. These each use the MaxPooling2D function which downsamples the input along its spatial dimensions. The result is then passed to a Dense layer with 100 neurons. This is a relatively small number chosen to limit overfitting, which is when a model performs well on training data, but poorly on test data. A Dropout layer is then used to randomly reset some of the neurons during training, a regularisation technique used to reduce overfitting by forcing the network to learn more generalisable features, rather than just memorising the test data. A second Dense layer is the output layer and contains 5 neurons to match the number of classification classes. This final layer uses the softmax activation function, which transforms the output into a probability distribution that shows which class the input is most likely to be a part of. This is particularly useful for multiclass classification.

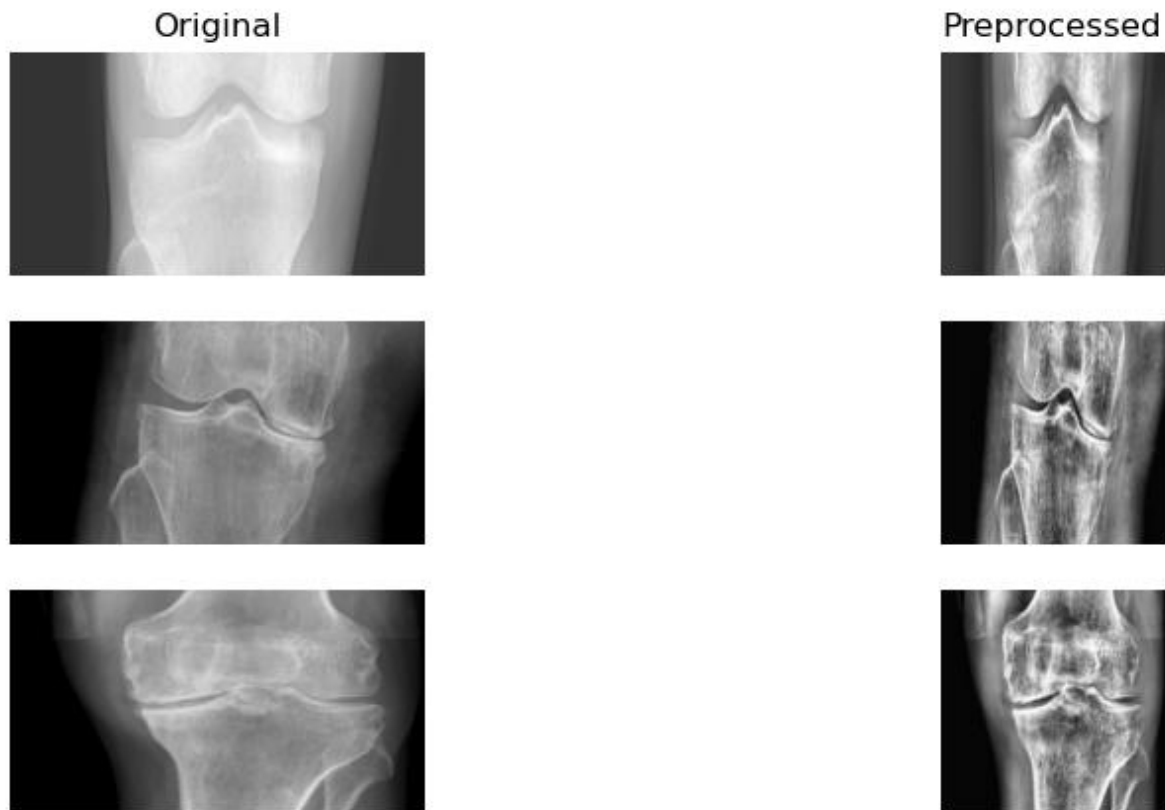
Transfer learning:



This starts with an input layer of size 224x224x3. The extra dimension accounts for the RGB colour values of the images that the VGG16 CNN was trained on. As the x-ray images are grayscale, an extra dimension was added to the data to be able to use the VGG16 convolutional base. After the convolutional base, the model uses GlobalAveragePooling2D to reduce the spatial dimensions while retaining the important features. The model then uses a Dense layer of 128 neurons and a Dropout layer before the final Dense (output) layer. The layers beyond the convolutional base act as a 5-class classifier specific to the project dataset.

Before entering the CNN, the data must be pre-processed. Pre-processing is an important part of machine learning, as it gives the models the best chance to identify features / patterns in the data by enhancing features and regularising image size. Some of the images in the dataset are at different resolutions, with the images of osteopenia being much higher resolution than the standard knee x-rays. To ensure all images are processed the same by the models, the resolution of all images was made to be 224x224. This reduces the features of the data, so makes detecting these features easier. This resolution was chosen to match the resolution used by the VGG16 CNN. This improves the shallow transfer learning model's performance and has little impact on the performance of the other models. To increase the contrast of the images, CLAHE (Contrast Limited Adaptive Histogram Equalisation) was used. This makes the features of the grayscale x-ray images more apparent, allowing the ML models to more easily identify features in the data.

Below is a comparison of some images before and after pre-processing.



To train and test the models, the data was split. This is so the model never sees the testing data when training. For the CNNs, the split was a 0.8 / 0.2 train test split, and a further 0.8 / 0.2 test validation split. This results in a 0.64 train / 0.16 validation / 0.2 test split. As the perceptron does not have a validation split, its ratio was simply 0.8 train / 0.2 test.

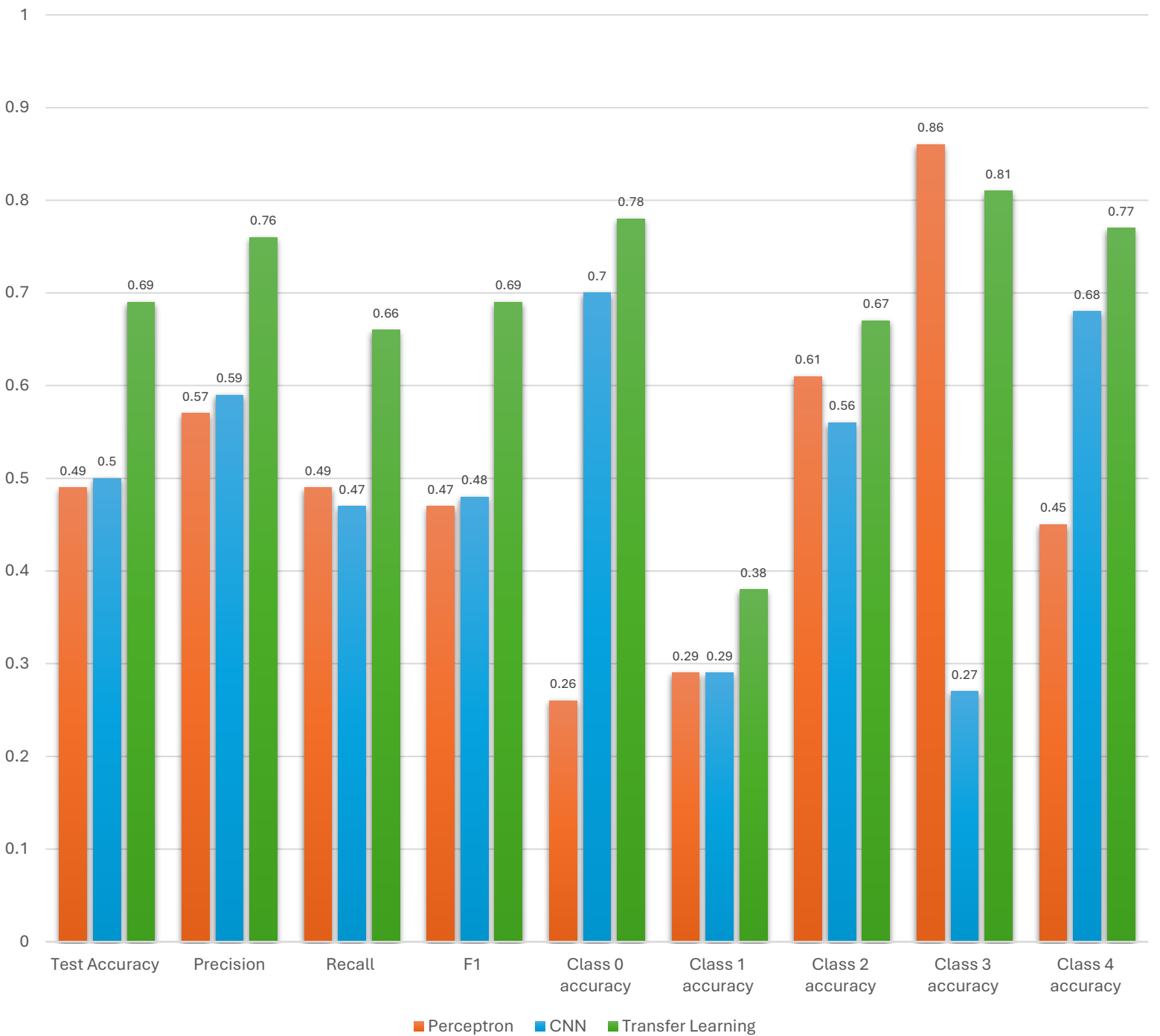
The perceptron was trained using a max iteration value of 1000, and an eta0 of 0.1.

The standard CNN was trained with hyperparameters focused on limiting overfitting. The number of epochs chosen was 50. While a high number, early stopping was implemented to limit overfitting by ending the training when the validation loss doesn't change significantly between epochs. The batch size selected was 16, a small number chosen due to the limited size of the dataset. The learning rate of the Adam optimiser was 1e-3, again focused on limiting overfitting / memorisation of the test data, and the validation split was 0.2, though this is 0.16 of the total data given the original 80 / 20 train test split. The number of classes was 5, corresponding to the 5 grades of knee injury in the dataset. The input shape was 224x224 to be consistent with the transfer learning model, and the loss was appropriately set to categorical crossentropy.

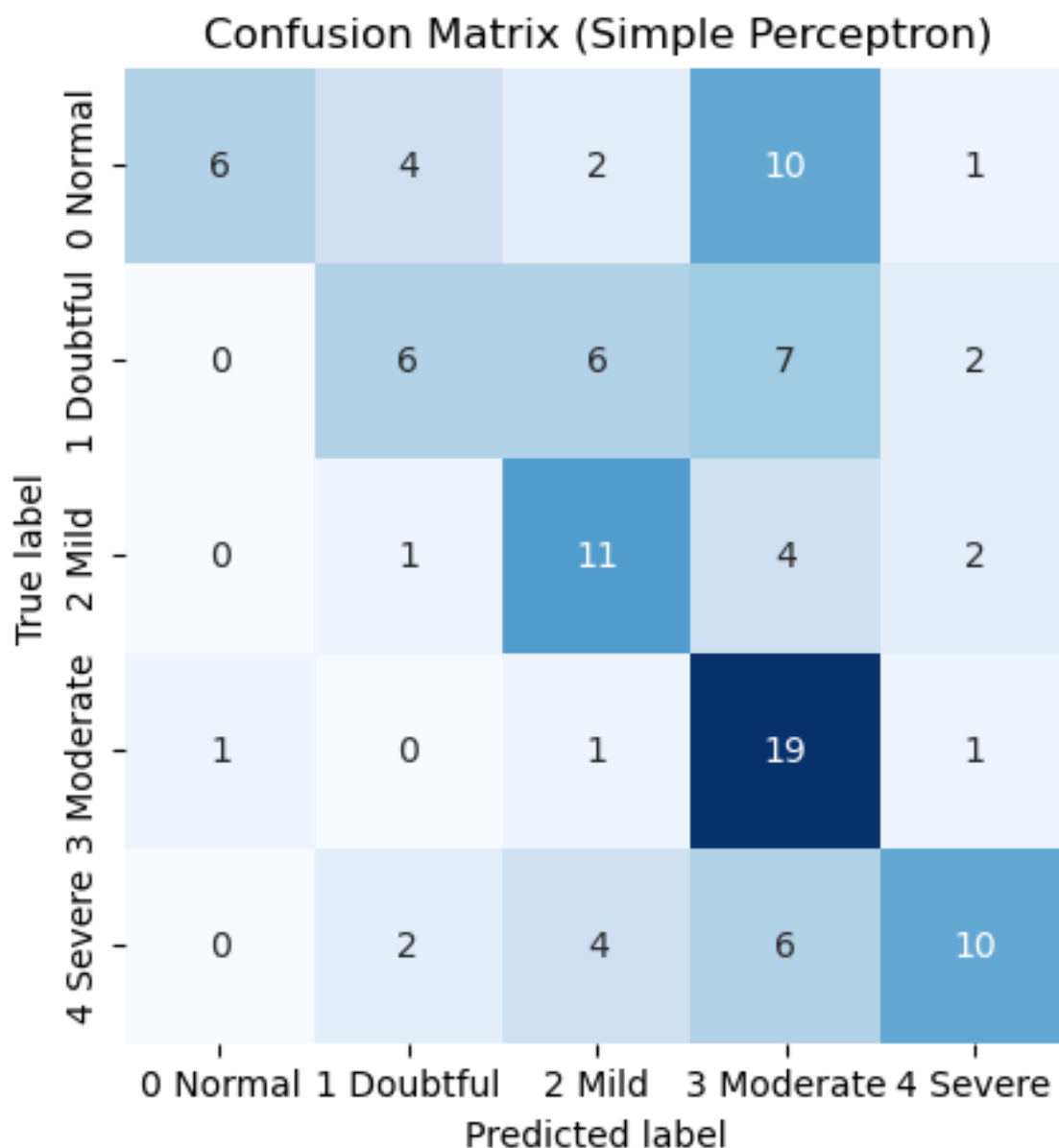
The shallow transfer learning model had 2 stages of training. In the first stage, the VGG16 convolutional base was frozen so only the specialised classifier was trained. This was done over 25 epochs with early stopping, using the Adam optimiser with a learning rate of 1e-3. After this first round of training, the convolutional base was then unfrozen, and a second shorter round of training was performed. This was to fine tune the convolutional base, so that it became more suited to the specific data used in the project. This used a lower learning rate of 1e-4 and only run for 10 epochs to ensure small, effective changes were made to the model.

Results

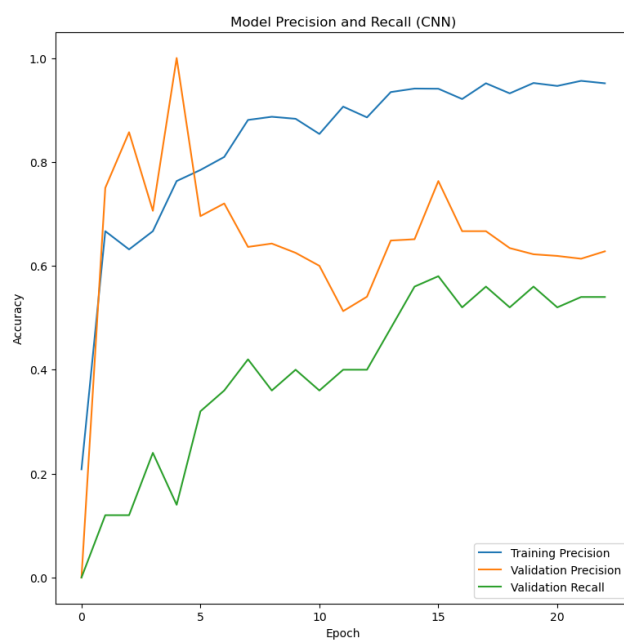
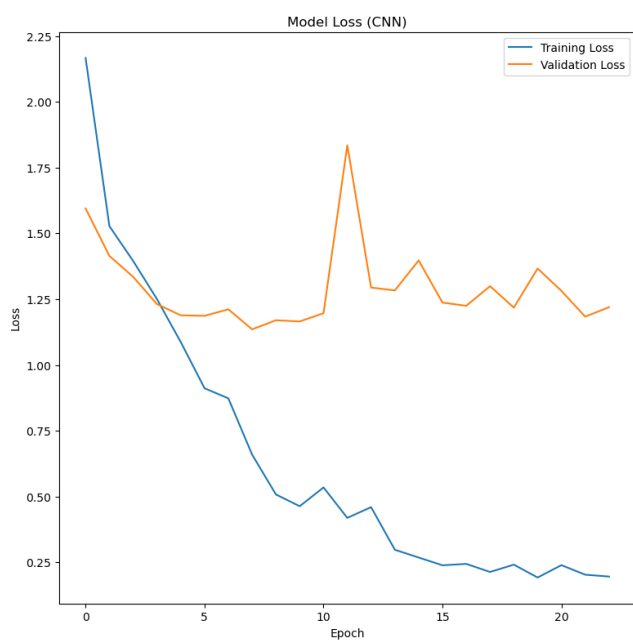
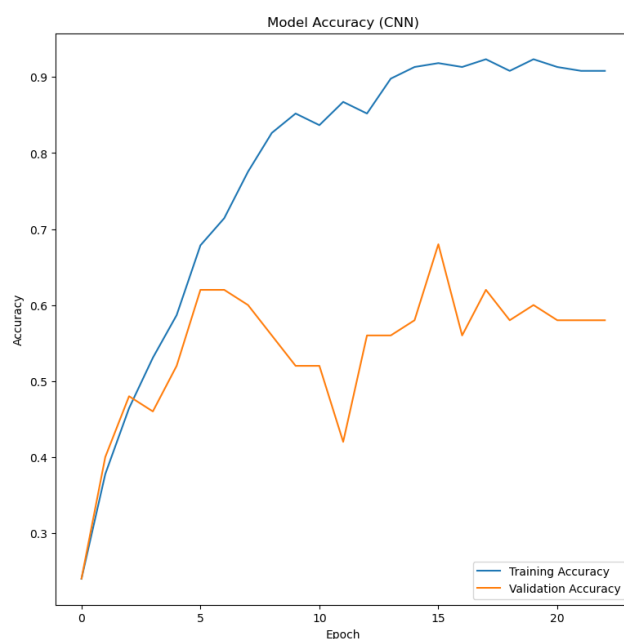
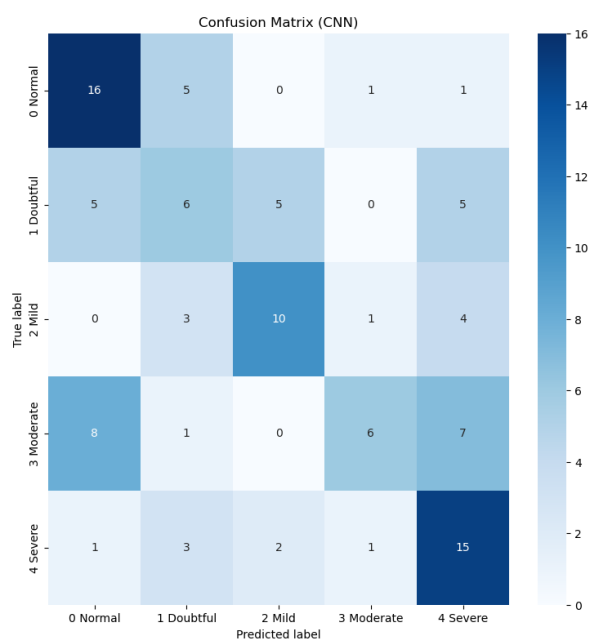
All three models were trained and evaluated on the dataset, measuring average accuracy, per class accuracy, precision, recall, and f1 score. Below is an overview of performance.



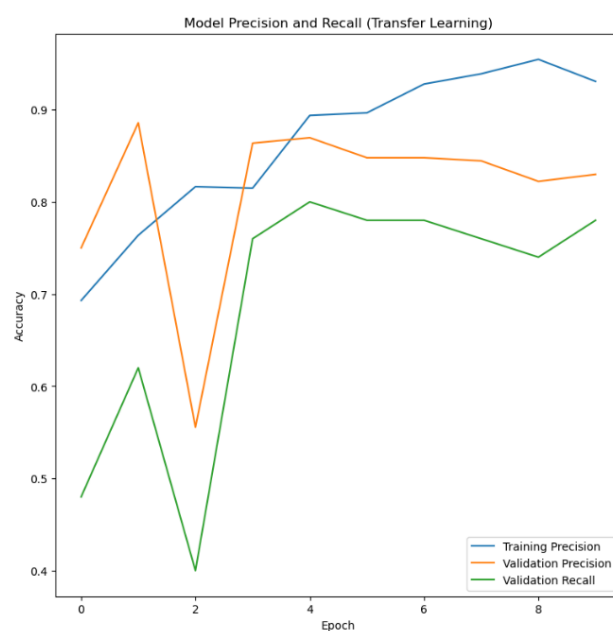
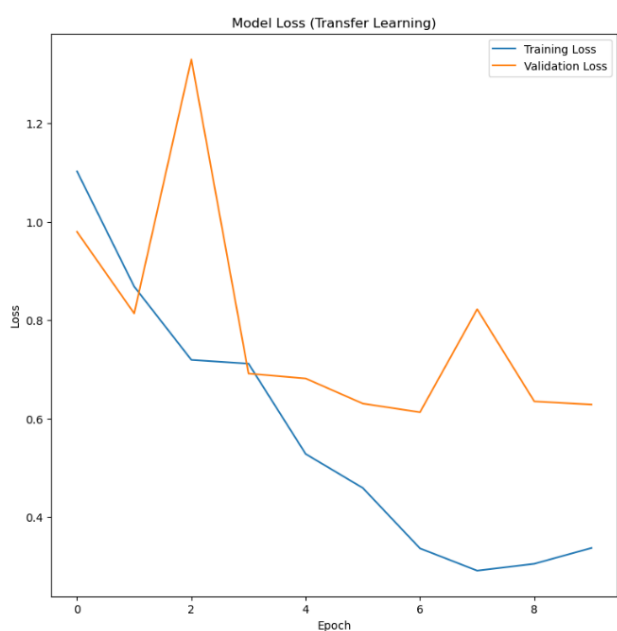
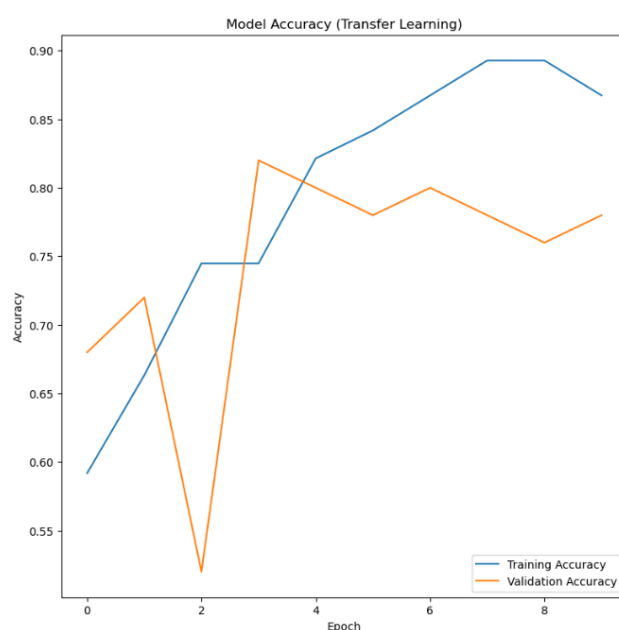
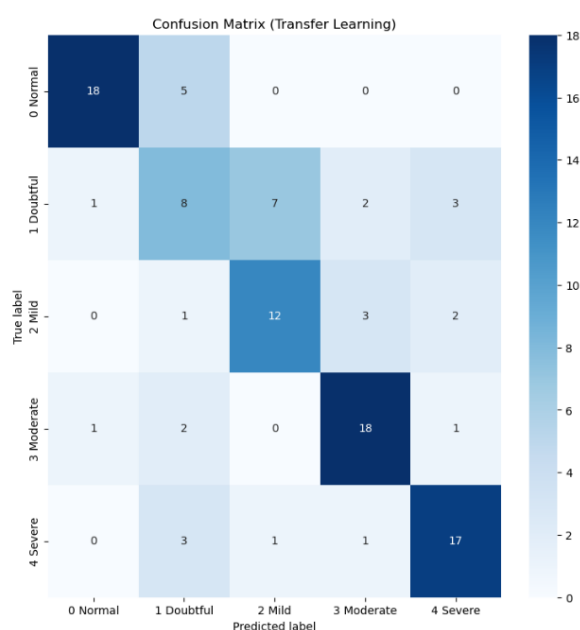
The perceptron achieved an accuracy of 0.49, the lowest of the three models. Its precision was measured at 0.57, recall at 0.49, and f1 at 0.47, lowest scores except in recall. The per class accuracies vary significantly. Classes 0,1,2,3,4 had accuracies of 0.26, 0.29, 0.61, 0.86, and 0.45 respectively. The low accuracy in classes 0 and 1 could be due to the small difference in features between a healthy knee and one only suffering from minor injury. The very high accuracy in class 3 could be explained by there being no images of osteopenia present in that dataset. This means the perceptron could focus on the features of the simple knee x-rays instead, resulting in high accuracy. This also explains how many of the incorrect classifications are class 3, as the simple knee x-rays from the other classes be easily misclassified. These results are also likely due to the data being too complex and having features that are hard to identify for a simple machine learning approach. Without the use of deep learning, identifying complex features from images accurately is difficult.



The CNN trained only on the project dataset performed poorly given its advanced deep learning techniques and higher computing requirements. It achieved an accuracy of 0.50, with precision of 0.59, recall of 0.47, and f1 of 0.48. This puts it barely ahead of the perceptron. In the Model Accuracy graph below, training accuracy becomes very high, but validation accuracy remains low. This indicates severe overfitting in the model, as it can correctly identify the classes of the training data, but struggles immensely with any new data. This observation is supported by the model producing a low training loss of 0.20, but a validation loss of 1.21 and testing loss of 1.71. These results indicate the dataset is far too small to train a CNN from scratch. The dataset contains 352 images, whereas a CNN requires tens of thousands of images to produce desirable results. The per class accuracy again varies. The accuracies produced were: 0: 0.70, 1: 0.29 , 2: 0.56, 3: 0.27, and 4: 0.68.



The shallow transfer learning model had by far the strongest performance out of the 3 models. It achieved a high test accuracy of 0.69, a precision of 0.76, a recall of 0.66, and an f1 of 0.69 . While the model does not suffer from overfitting to the same extent as the from scratch CNN, the higher training accuracy of 0.89 shows that it is still occurring. This is again due to the small size of the dataset provided. Further evidence for overfitting can be seen in the loss metric. The model produced a training loss of 0.34, but a testing loss of 0.87. While a much better score than the from-scratch CNN, the discrepancy still indicates overfitting. The per class accuracy of the model is much higher than the others. For classes 0,1,2,3,4 the accuracies were measured at 0.78, 0.38, 0.67, 0.81, and 0.77. The lower accuracy for class 1 could be due to there being similar features between class 1 and 2, as the majority of the misclassification of grade 1 images identify them as grade 2.



Summary

The main challenge is the limited size of the dataset. There are only 352 datapoints, which makes training an ML model difficult and is the cause of most of the inaccuracies and overfitting. The images themselves also create issues. In all classes but class 3, there are higher resolution images of osteopenia, which could have different features to other knee injuries, which are then given in lower image resolutions. In addition, there are some images that contain both knees. The additional complexity of identifying features on both types of images can cause inaccuracies in the models.

To improve the project and tackle the main issue being faced, more data must be collected. For a CNN to be effective, a dataset containing around 10,000 images is needed. In addition, this project was performed without the use of some data augmentation techniques such as rotating and inverting the images. This can increase the number of datapoints for the models to use in training.

The project has found that the CNN using shallow transfer learning with fine tuning is the highest performing model out of the 3 selected for experimentation. This is due to the well trained VGG16 convolutional base it uses which limits the impact of the small dataset on the performance of the model.

(1998 words)

References

<https://www.geeksforgeeks.org/machine-learning/how-to-plot-confusion-matrix-with-labels-in-sklearn/>

<https://www.geeksforgeeks.org/python/clahe-histogram-equalization-opencv/>

<https://www.geeksforgeeks.org/deep-learning/evaluation-metrics-in-tensorflow/>

<https://link.springer.com/article/10.1007/s11042-023-16016-2>

<https://www.geeksforgeeks.org/deep-learning/sklearn-classification-using-perceptron/>

<https://www.geeksforgeeks.org/deep-learning/transfer-learning-fine-tuning-using-keras/>

<https://towardsdatascience.com/cnn-transfer-learning-fine-tuning-9f3e7c5806b2/>

<https://www.nature.com/articles/s41598-025-34806-6>

<https://www.geeksforgeeks.org/deep-learning/training-and-validation-loss-in-deep-learning/>

<https://blog.finxtter.com/5-best-ways-to-evaluate-a-cnn-model-using-tensorflow-with-python/>

<https://www.geeksforgeeks.org/deep-learning/softmax-vs-sigmoid-activation-function/>

<https://www.sciencedirect.com/science/article/pii/S2772662224000742>

<https://www.geeksforgeeks.org/machine-learning/hyperparameter-tuning-fixing-overfitting-in-neural-networks/>

<https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-tensorflow/>

<https://www.geeksforgeeks.org/machine-learning/visualizing-representations-of-outputs-activations-of-each-cnn-layer/>

https://www.tensorflow.org/tutorials/images/transfer_learning

<https://www.geeksforgeeks.org/computer-vision/vgg-net-architecture-explained/>

<https://www.geeksforgeeks.org/computer-vision/difference-between-traditional-computer-vision-techniques-and-deep-learning-based-approaches/>

<https://chatgpt.com/share/69b72529-5620-8013-89fd-49177f297d3a>